ACME: Agentic Contract-Based Multilevel Engineering for High-Assurance Complex Systems

PI: Pierluigi Nuzzo, Associate Professor, Department of Electrical Engineering and Computer Sciences, University of California, Berkeley

Cash funding needed: $80,000
AWS Promotional Credits needed: $25,000–$40,000

Amazon contact (optional): [Amazon contact name], [email]

Abstract - Large language model (LLM) agents are increasingly employed to design complex systems, such as register-transfer level (RTL) hardware and controllers for teams of robots. However, AI-generated designs are error-prone: agents decompose problems incorrectly, drift from their instructions over long design tasks, and leave no record that allows an error to be traced back to its cause. Decomposition is also hard in itself, as distributed reactive synthesis is undecidable in general. In this project, we propose a framework, termed Agentic Contract-Based Multilevel Engineering (ACME), for sound and traceable decomposition of complex systems based on assume-guarantee (A/G) contracts. In ACME, agents propose contracts at every level of a design hierarchy, and a contract engine with model checking and synthesis back ends certifies each step. ACME follows a three-pronged approach: an extension of contract theory with decidable classes of decomposition, sound approximations of the contract quotient, and circular reasoning; an agentic decomposition search in which LLM agents, guided by a library of certified components, propose contracts and only the leaf verifiers change across domains; and learning from formal feedback, in which counterexamples and unrealizable cores repair decompositions in context and serve as verifiable rewards to post-train the models that propose them. ACME will be built on CHASE, our open-source contract-based design framework, and evaluated primarily on hardware designs and on multi-robot missions.

Keywords – assume-guarantee contracts, compositional verification, reactive synthesis, GR(1), neurosymbolic reasoning, LLM agents, reinforcement learning with verifiable rewards, hardware formal verification, autoformalization.

Introduction

LLM agents are increasingly employed to design complex systems, from custom silicon to controllers for fleets of autonomous robots and workflows of cooperating software agents. Agents generate hardware assertions [1, 2], program invariants [3], and code faster than design teams can review them. However, AI-generated designs are error-prone, and the errors concentrate in decomposition, which every complex design requires. In our experience, agents that decompose end-to-end hardware properties may report timed-out runs as proofs or rely on circular assumptions across lemmas. Over long design tasks, agents that delegate subtasks or compact their context restate the requirements in natural language and drift away from them [4]. Moreover, the reasoning behind a design is buried in transcripts, so a late error cannot be traced to its cause. As a result, AI-driven design of complex systems relies today on trial and error and on testing the final artifact. A new generation of design methods would be essential to make every step of AI-driven design traceable and checkable, and to keep the overall design correct as it is decomposed across components, agents, and abstraction levels.

Contract-based design provides a formal basis for this goal. An A/G contract specifies what a component guarantees under assumptions on its environment [5]. A decomposition is correct when the composition of the component contracts refines the system contract, and the algebra of contracts provides operators, such as the quotient, that compute the weakest specification a missing component must satisfy [6, 7]. Existing contract-based tools, including CHASE [8], Pacti [9], and OCRA [10], assume that contracts are written by human designers, and existing decomposition methods for synthesis rely on a fixed library of

components [11]. Decomposition is also used by recent LLM reasoning methods: language models search over decompositions [12], compose functions validated by tests [13], and delegate subgoals to automated provers [14, 15, 16]. However, their checks are either informal, such as tests and verbal reflection [17], or limited to proof goals, and do not establish that the components of a system are realizable and compose correctly. Letting agents write the contracts raises three scientific questions. First, the contract operators must remain computable within the fragments that verification and synthesis tools accept; for instance, the quotient of two GR(1) contracts in general falls outside GR(1). Second, the existence of a decomposition into realizable components is undecidable for general architectures [18], so the search over decompositions must be organized such that certified progress is never lost. Third, an agent may search indefinitely unless each failure is explained in a form that excludes whole families of candidates and improves its future proposals.

We propose ACME to address these challenges through three thrusts. The central principle of ACME is that agents propose and contracts certify: a decomposition step proposed by an agent is accepted only with a certificate that a solver can check, so that guarantees established at higher abstraction levels propagate consistently to the implementation. Thrust 1 develops decidable, computable contract decomposition within synthesis fragments. Thrust 2 develops multilevel agentic decomposition guided by a certified component library, with domain-specific leaf verifiers. Thrust 3 learns from formal feedback, both in context and through post-training with engine verdicts as verifiable rewards. ACME builds on our prior work on contract-based design methodologies [19] and the CHASE framework [8], which provides a contract algebra with interfaces to verification and synthesis tools and to LLM agents. Our work on contract-based architecture exploration [20, 21] has shown that infeasibility certificates can be generalized via subgraph isomorphism to prune large design spaces, and received the Best Paper Award at DATE 2024. ACIES [22] applies this principle to multi-robot control, and our ongoing work to RTL verification. In a word, ACME will advance AI-driven hardware, robot, and workflow design through solver-checkable certificates for decomposition steps, traceable errors, and reuse of certified components.

Methods

The core of ACME is a neurosymbolic loop: LLM agents propose decompositions (Thrust 2), a contract engine certifies them on the theory of Thrust 1, and its verdicts return to the agents, in context and as rewards for post-training (Thrust 3). Hardware is the main testbed, multi-robot control the second domain, and multi-agent workflow design an additional example.

Thrust 1: Contract Theory for Decomposition. Hardware contracts relate the ports of register-transfer-level modules over clock cycles and may refer to ghost state verified on the design. Robot contracts couple a GR(1) mission layer [23] with controllers certified by control barrier functions [24] or reachability; learned policies, such as vision-language-action models [25], are guarded at run time by safety filters and monitors that report violated assumptions to the decomposition. For workflows, an linear temporal logic on finite traces (LTL_f) contract [26] specifies the admissible tool calls of a role. The existence of a decomposition into realizable components is undecidable in general [18]; we will identify decidable classes, starting from pipelines. Since the quotient of two GR(1) contracts in general leaves the fragment, we will develop approximations within GR(1) that strengthen its guarantees, so that any controller synthesized for them is correct for the composition. For circular dependencies in pipelines, handshakes, and roles that wait on each other, we will adopt a circular A/G rule restricted to safety guarantees with an inductive side condition [27, 28].

Thrust 2: Agentic Decomposition Search. ACME represents a design as a directed acyclic graph of contracts, built recursively by LLM agents. At an internal node, an agent receives the parent contract, the interfaces of the subcomponents, and certified decompositions of similar components retrieved from the library, and proposes one contract for each subcomponent, formalizing natural-language requirements where needed. The proposal is accepted only if the component contracts are consistent and non-vacuous, bounded checks on the actual components find no counterexample, and their composition refines the parent contract. Each

subcomponent is then discharged by a back end or handed to a new agent whose context contains only its own contract and therefore does not grow with the design; a node that cannot satisfy its contract returns a witness to its parent, which revises the decomposition while certified subtrees are kept. Alternative decompositions are explored by a tree search in which the LLM acts as the policy and the engine prunes candidates. Only the leaves change across domains: a hardware module proved by a model checker, a robot controller certified by a synthesis or reachability tool, or a learned policy guarded at run time. As an example, the same procedure designs multi-agent workflows before deployment: the quotient gives the weakest contract of each role, and the contracts, which fix the tools a role may call, also partition the key-value cache into modules aligned with contract states [29, 30], prefetched for reachable states to reduce the serving cost.

Thrust 3: Learning from Formal Feedback. The engine separates a formalism-independent contract algebra, checks reduced to validity or satisfiability, and formalism-specific analyses such as unrealizable cores, vacuity [31], and well-separation [32]; every failed check returns a counterexample or a minimal core that points to the responsible terms. A Model Context Protocol (MCP) server and an agent skill expose these operations to agents, while no LLM is part of the engine. Refinement failures are first repaired with the quotient, before any agent call; otherwise, the counterexample is returned to the agent in context as a trace over the responsible contract terms. As learned clauses prune the search of a SAT solver, the core of a failed decomposition is generalized via subgraph isomorphism [20] into a constraint that excludes every candidate assigning an isomorphic component the same terms. Certified contracts are stored with their components in a library that grows with each design, and whose coverage is measured by injecting faults. Beyond in-context feedback, certified decompositions, together with the module hierarchies of existing designs as weak supervision, are used for supervised fine-tuning of an open-weight proposer; reinforcement learning then uses the checks of the engine as verifiable rewards, graded by the consistency, refinement, and realizability of each component contract, so that partially correct decompositions still provide signal. Since every proposal is still checked by the engine, soundness does not depend on the proposer.

Evaluation and preliminary results. Our prior work combines diverse candidate generation, solver-based screening, and auditable agent execution to improve auto-formalization reliability [33, 34][1]. These methods underpin contract generation, checking, and feedback-guided revision in ACME. In hardware, component contracts are SystemVerilog assertions (SVAs) with ghost-state models, discharged by hardware model checkers. Our prototype certified an end-to-end in-order delivery property of a HierSVA asynchronous link with 25 model-checking runs and about 16,000 LLM input tokens, while direct model checking of the end-to-end property did not finish within a considerably longer time limit; the decomposition found at FIFO depth 8 certified depths 16 to 512 with no further LLM calls, and an agent without the protocol used 24 times as many tokens without completing the proof. In multi-robot control, ACIES [22] decomposes mission requirements into GR(1) contracts, synthesizes controllers, and executes them under control barrier functions with runtime monitors; in a three-level decomposition with 42 nodes, the contract library closed 26 of the 39 leaves. We will compare frontier models with and without formal feedback and retrieval against the post-trained proposer, measuring the requirements certified beyond direct verification or synthesis, the fraction of proposals accepted, and the solver runs and tokens per certificate.

Expected results

Months	Research result	Deliverable
1–3	Decomposition agent with in-context formal feedback; GR(1) quotient approximations	CHASE release with MCP server and agent skill; technical report

[1] `https://github.com/zyni2001/draft-and-prune`

4–6	Library retrieval and learned constraints; decidable classes; circular A/G rule	RTL benchmark with faulty variants; theory paper (e.g., CAV)
7–9	Agentic decomposition search on hardware and robots; dataset of certified decompositions	Evaluation; open dataset; paper (e.g., FMCAD)
10–12	Proposer post-trained with fine-tuning and reinforcement learning on engine rewards	Open-weight proposer; paper (e.g., NeurIPS); tutorial; talks to Amazon

Beyond the first year, the certified decompositions will grow into a training corpus for a decomposition model shared across domains; the benchmark, dataset, and tutorial will be released to the automated reasoning and machine learning communities, together with all code and run traces.

Funds needed

Cash funding ($80,000). Partial support of one graduate student researcher (stipend and fees, [amount]), and travel to present the results at conferences ([amount]). No indirect expenses are included.

AWS Promotional Credits ($25,000–$40,000). Amazon Bedrock for agent inference, which our pipelines already use; Amazon SageMaker GPU instances for post-training an open-weight decomposition proposer (Thrust 3); Amazon EC2 compute-optimized instances for parallel runs of nuXmv, slugs, and SymbiYosys, which also compute the rewards during post-training; and Amazon S3 for benchmarks and run traces. Commercial verification tools use campus licenses.

Additional information

1. Application domain and analysis techniques. Compositional design of hardware and multi-robot systems, with multi-agent LLM workflows as an additional example, using A/G contracts, linear temporal logic (LTL) and LTL$_f$ model checking (nuXmv), satisfiability modulo theories (SMT) reasoning, GR(1) synthesis (slugs), and hardware model checking (e.g., SymbiYosys). It addresses the call topics on sound neurosymbolic reasoning, LLM–solver interfaces, vacuity and inconsistency detection, and formal models of hardware.

2. Current applications. CHASE has been used for requirement formalization and validation of cyber-physical systems [8], contract-based architecture exploration [20, 21], and consistency and availability analysis of distributed systems in collaboration with Lockheed Martin [35]. Our SMC solver [36] has also enabled counterexample-guided repair of neural networks [37]. Our RTL flow runs on OpenTitan and HierSVA designs.

3. Potential applications to Amazon. Verification of AWS silicon; controllers for Amazon Robotics fleets; design of multi-agent workflows on Amazon Bedrock whose decomposition into roles is certified before deployment, at lower serving cost; and consistency and vacuity analysis of formalized policies, complementary to Automated Reasoning checks in Bedrock Guardrails.

4–5. Prior Amazon Research Awards. No; the project team has received no Amazon funding or AWS credits in the past three years.

6. Assumptions and soundness. A certified result is sound when (i) the back ends return correct verdicts; (ii) the translation into each back end preserves the semantics of the contracts; (iii) every composition step is acyclic or satisfies the side condition of the circular rule in Thrust 1; and (iv) the top-level specification is correct, which is why agent-written contracts are checked for consistency and vacuity. Bounded results never count as proofs, LLM outputs are never trusted, and quotient approximations only strengthen guarantees.

7. Tools. *License:* BSD 3-Clause for CHASE and its MCP server, Apache 2.0 for the RTL flow, and MIT for ACIES. *On-boarding:* a website with a usage guide and API reference, a Docker image, and the agent skill; a tutorial is planned. *Maintenance:* maintained by our lab; over 90 commits in three months.

Appendix A – Reference

[1] Orenes-Vera, Marcelo, et al. “AutoSVA: Democratizing Formal Verification of RTL Module Interactions.” DAC. 2021.

[2] Kang, Minwoo, et al. “FVEval: Understanding Language Model Capabilities in Formal Verification of Digital Hardware.” arXiv:2410.23299. 2024.

[3] Wu, Haoze, Clark Barrett, and Nina Narodytska. “Lemur: Integrating Large Language Models in Automated Program Verification.” ICLR. 2024.

[4] Cemri, Mert, et al. “Why Do Multi-Agent LLM Systems Fail?” arXiv:2503.13657. 2025.

[5] Benveniste, Albert, et al. “Contracts for System Design.” Foundations and Trends in Electronic Design Automation. 2018.

[6] Incer, Inigo, Albert Benveniste, and Alberto Sangiovanni-Vincentelli. “Some Algebraic Aspects of Assume-Guarantee Reasoning.” arXiv:2309.08875. 2023.

[7] Incer, Inigo. “The Algebra of Contracts.” Ph.D. thesis, UC Berkeley, EECS-2022-99. 2022.

[8] Nuzzo, Pierluigi, et al. “CHASE: Contract-Based Requirement Engineering for Cyber-Physical System Design.” DATE. 2018.

[9] Incer, Inigo, et al. “Pacti: Scaling Assume-Guarantee Reasoning for System Analysis and Design.” arXiv:2303.17751. 2023.

[10] Cimatti, Alessandro, Michele Dorigatti, and Stefano Tonetta. “OCRA: A Tool for Checking the Refinement of Temporal Contracts.” ASE. 2013.

[11] Iannopollo, Antonio, Stavros Tripakis, and Alberto Sangiovanni-Vincentelli. “Specification Decomposition for Synthesis from Libraries of LTL Assume/Guarantee Contracts.” DATE. 2018.

[12] Yao, Shunyu, et al. “Tree of Thoughts: Deliberate Problem Solving with Large Language Models.” NeurIPS. 2023.

[13] Zelikman, Eric, et al. “Parsel: Algorithmic Reasoning with Language Models by Composing Decompositions.” NeurIPS. 2023.

[14] Jiang, Albert Q., et al. “Thor: Wielding Hammers to Integrate Language Models and Automated Theorem Provers.” NeurIPS. 2022.

[15] Yang, Kaiyu, et al. “LeanDojo: Theorem Proving with Retrieval-Augmented Language Models.” NeurIPS. 2023.

[16] Ren, Z. Z., et al. “DeepSeek-Prover-V2: Advancing Formal Mathematical Reasoning via Reinforcement Learning for Subgoal Decomposition.” arXiv:2504.21801. 2025.

[17] Shinn, Noah, et al. “Reflexion: Language Agents with Verbal Reinforcement Learning.” NeurIPS. 2023.

[18] Pnueli, Amir, and Roni Rosner. “Distributed Reactive Systems Are Hard to Synthesize.” FOCS. 1990.

[19] Nuzzo, Pierluigi, et al. “A Platform-Based Design Methodology With Contracts and Related Tools for the Design of Cyber-Physical Systems.” Proceedings of the IEEE. 2015.

[20] Xiao, Yifeng, et al. “Efficient Exploration of Cyber-Physical System Architectures Using Contracts and Subgraph Isomorphism.” DATE. 2024.

[21] Xiao, Yifeng, and Pierluigi Nuzzo. “Contract-Based Architecture Exploration of Cyber-Physical Systems via Satisfiability Modulo Convex Programming.” DATE. 2026.

[22] Xiao, Yifeng, et al. “ACIES: A Self-Evolving Agentic Contract-Based Framework for Cyber-Physical System Control Synthesis.” REALM Workshop at EMNLP. 2026.

[23] Bloem, Roderick, et al. “Synthesis of Reactive(1) Designs.” Journal of Computer and System Sciences. 2012.

[24] Ames, Aaron D., et al. “Control Barrier Functions: Theory and Applications.” ECC. 2019.

[25] Kim, Moo Jin, et al. “OpenVLA: An Open-Source Vision-Language-Action Model.” CoRL. 2024.

[26] De Giacomo, Giuseppe, and Moshe Y. Vardi. “Linear Temporal Logic and Linear Dynamic Logic on Finite Traces.” IJCAI. 2013.

[27] McMillan, Kenneth L. “Circular Compositional Reasoning about Liveness.” CHARME. 1999.

[28] Namjoshi, Kedar S., and Richard J. Trefler. “On the Completeness of Compositional Reasoning.” CAV. 2000.

[29] Gim, In, et al. “Prompt Cache: Modular Attention Reuse for Low-Latency Inference.” MLSys. 2024.

[30] Yao, Jiayi, et al. “CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion.” EuroSys. 2025.

[31] Maoz, Shahar, and Rafi Shalom. “Inherent Vacuity for GR(1) Specifications.” ESEC/FSE. 2020.

[32] Maoz, Shahar, and Jan Oliver Ringert. “On Well-Separation of GR(1) Specifications.” FSE. 2016.

[33] Ni, Zhiyu, Yifeng Xiao, and Zheng Liang. “Agentified Assessment of Logical Reasoning Agents.” Agents in the Wild (AIWILD) Workshop at ICLR. 2026.

[34] Ni, Zhiyu, et al. “Draft-and-Prune: Improving the Reliability of Auto-formalization for Logical Reasoning.” arXiv:2603.17233. 2026.

[35] Xiao, Yifeng, et al. “Contract-Based Consistency and Availability Analysis for Distributed Cyber-Physical Systems.” MEMOCODE. 2026.

[36] Shoukry, Yasser, et al. “SMC: Satisfiability Modulo Convex Programming.” Proceedings of the IEEE. 2018.

[37] Fayyazi, A., et al. “Efficient Counterexample-Guided Fairness Verification and Repair of Neural Networks Using Satisfiability Modulo Convex Programming.” IJCAI. 2025.